CareTrace: A Neurosymbolic Approach to Pediatric After-Hours Triage

DATASCI 290 — Neurosymbolic AI, UC Berkeley School of Information

Kirthi Shanbhag, Niyanthri Naresh, Sohail Khan, XinRan Zhang

April 13, 2026

Code: https://github.com/kirthistaank/CareTrace

Introduction

After-hours pediatric fever is one of the most common reasons families seek urgent guidance. Parents must decide—often alone at night—whether to go to the ER, see a doctor in the morning, or manage at home. Existing tools fail at this: symptom checkers are static and one-directional, while LLM assistants can change their reasoning mid-conversation, fabricate dosing, or downgrade urgency based on irrelevant context. Both patterns are unsafe for triage.

CareTrace is a neurosymbolic prototype that addresses these failure modes. It combines LLM-based interpretation of caregiver language with a SNOMED CT knowledge graph for clinical grounding, deterministic PyDatalog triage rules derived from the Seattle Children's Fever CPG, and provenance-annotated explanation generation. The scope is deliberately bounded: febrile illness, GI symptoms, and dehydration risk in children under 12, with three possible dispositions: **ER now**, **urgent same-day**, or **home management tonight**.

This report describes the architecture, knowledge graph design, rule design, system behavior, evaluation against a vanilla LLM baseline, and an extra investigation into Logic-Augmented Generation (LAG).

1 Architecture

1.1 How Neural + KG + Rules Interact

Each caregiver turn flows through a five-node LangGraph state machine:

```
interpret -> kg -> safety -> [explain | explain_incomplete] -> END
```

Neural layer (Agents). The `interpret` node extracts structured clinical fields (`CaseFields`) from free-text caregiver messages using either an LLM (GPT-4o-mini with Pydantic structured output) or a deterministic heuristic fallback (regex + keyword matching). A `_merge_non_empty` strategy accumulates evidence across turns without overwriting prior values.

Knowledge Graph layer. The `kg` node maps structured case fields and raw text to SNOMED CT search terms, then queries a Neo4j AuraDB mini-KG for matching concepts and `IS_A` ancestors. The KG annotates caregiver mentions with standard terminology (e.g., `alertness=altered` → Lethargy [SNOMED 214264003]) and provides semantic hierarchy context for explanation grounding. The KG does not independently determine disposition—that is the exclusive role of the symbolic rules.

Symbolic Rules layer. The `safety` node first enforces a 5-field required intake policy. If fields are missing, the pipeline routes to `explain_incomplete`. If complete, `evaluate_triage()` asserts session-scoped PyDatalog facts and evaluates deterministic predicates in strict priority: `er_now` → `urgent_same_day` → `home_candidate`. Output: a `TriageDecision` with disposition, fired rule IDs, and medication safety flags.

Explanation layer. The `explain` node synthesizes the decision into a clinician-style response. Three modes: deterministic templates, LLM explanation, and Logic-Augmented Generation (LAG—see Section 6).

1.2 Orchestration Flow

```
Caregiver text --> [interpret] --> [kg] --> [safety] --+
                                                       |
                              +------------------------+------------------+
                              v                                           v
                        missing fields?                           all fields present
                              |                                           |
                     [explain_incomplete]                              [explain]
                     (ask follow-ups)                             (issue disposition)
```

Figure 1: LangGraph orchestration: each turn triggers interpret → kg → safety → explain/incomplete.

2 Knowledge Graph Design

2.1 Concepts and Rationale

The KG is a SNOMED CT subgraph seeded from the Seattle Children's Fever CPG. We extracted "Call the doctor if your child..." bullet points from the CPG, mapped each to Snowstorm queries, resolved SCTIDs, and loaded into Neo4j AuraDB.

Table 1: Seed concepts from CPG extraction.

CPG Criterion	SCTID	Preferred Term	Rule-Driving?
Fever threshold	386661006	Fever	Yes (anchor)
Seizure	91175000	Seizure	Yes
Breathing difficulty	267036007	Dyspnea	Yes
Fast breathing	271823003	Tachypnea	Yes
Reduced consciousness	214264003	Lethargy	Yes
No urination 8 h	83128009	Oliguria	Yes
Dehydration signs	34095006	Dehydration	Yes
Vomiting often	422400008	Vomiting	Yes

Four additional CPG criteria (rash, neck pain, otalgia, sore throat) are present in the KG for annotation and future rule extension but not yet encoded as triage predicates.

Schema: `(:Concept {sctid, pt, kg_source}) -[:IS_A]-> (:Concept)` and `-[:REL {typeId, typeTerm}]-> (:Concept)`.

2.2 Bounded Expansion

BFS from 12 seeds + anchor, following `IS_A` parents + up to 3 attribute neighbours, capped at **45 total concepts**. This keeps the graph clinically focused (e.g., Tachypnea → IS_A → Abnormal respiratory rate → IS_A → Finding of respiratory system).

2.3 Runtime Retrieval

Each turn: `kg_mentions_from_case_and_text` maps fields + raw text to search terms → `find_concepts_by_term` runs case-insensitive `CONTAINS` on `pt` → `ancestors_via_is_a` traverses up to 8 hops. This is a scoped prototype retrieval method adequate for exact CPG-term matching.

3 Rule Design

3.1 Key Rules and Escalation Logic

Table 2: Hard ER gates (any single trigger → ER_NOW).

Rule ID	Condition	CPG Basis
`R_ER_ALERTNESS`	alertness = altered	Not alert → ER
`R_ER_BREATHING`	breathing = distress	Trouble breathing → ER
`R_ER_DEHYDRATION_SEVERE`	dehydration_severe = yes	No urine 8 h + poor intake → ER
`R_ER_NO_FLUID_NO_URINE`	fluid = none AND urine = no	Same composite, overlap for auditability
`R_CPG_SEIZURE`	seizure = yes	Febrile convulsion → ER
`R_CPG_INFANT_UNDER_3MO`	age $<$ 3 mo AND temp $>$ 100.4°F	Infant fever → ER

R_ER_DEHYDRATION_SEVERE and R_ER_NO_FLUID_NO_URINE intentionally overlap (derived fact vs. raw fields) so reviewers see the gate from two angles.

Table 3: Urgent same-day gates (trigger + NOT ER → URGENT_SAME_DAY).

Rule ID	Condition
R_URGENT_VERY_HIGH_FEVER	temp $\geq$ 104°F
R_URGENT_REPEATED_VOMIT_POOR_FLUID	vomiting = repeated AND fluid = poor
R_URGENT_TACHYPNEA_CONCERN	breathing = tachypnea_concern
R_URGENT_FEVER_OVER_3_DAYS	fever duration $\geq$ 72 h

Home Management: R_HOME_CONSERVATIVE fires when alertness is normal/sleepy_ok, breathing is normal, fluid intake is anything other than `none`, and neither ER nor urgent gates fired.

3.2 Derived Facts

`_case_to_facts()` converts raw fields to rule-ready facts:

- **Temperature bucketing:** $\geq$104°F → `very_high`, $\geq$103°F → `high`, else `non_extreme`. Example: `temp_f=101.8` → `cf(S, "temp_f", "non_extreme")`.
- **Dehydration composite:** poor fluid AND no urine → `severe`; either one → `risk`.
- **Age gates:** < 6 months → ibuprofen contraindicated; < 3 months with fever → CPG infant gate.

3.3 Required Intake Policy

Five fields must be present before any disposition: temperature (or `temp_unknown`), alertness, breathing, fluid intake, urine last 8 h. Until all five are present, the system asks follow-up questions rather than issuing a disposition.

3.4 Medication Safety Flags

Independent of disposition, `med_flag` predicates surface: NSAID caution with dehydration risk, antibiotic interaction review, ibuprofen age gate (< 6 mo), and antipyretic restriction (< 3 mo). The template explanation includes provenance strings citing the Seattle Children's CPG URL.

4 System Behavior: Example Runs

4.1 Scenario 1 — Home Management (6 yo, 101.8°F)

Turn	Caregiver	Extracted	Action
1	"6-year-old has a fever, threw up once, looks wiped out."	age=6 y, vomiting=once	Missing 4/5 → asks
2	"Temp 101.8. Tired but answers me. No breathing issues. Sipping water."	temp=101.8, alertness=sleepy_ok, breathing=normal, fluid=some	Missing: urine → asks
3	"On amoxicillin. Vomited once. Peed earlier this evening."	meds=[amoxicillin], urine=yes	All 5 → evaluate

Fired: ✓ R_HOME_CONSERVATIVE. **Not fired:** all 6 ER gates, all 4 urgent gates.
Disposition: HOME_MANAGEMENT + safety netting + med flag: `antibiotic_on_file_review_interactio`

4.2 Scenario 2 — ER Now (6 yo, 103.5°F, barely responding)

Turn	Caregiver	Key Extractions	Action
1	"6-year-old has a fever, threw up, looks wiped out."	age=6 y	Missing → asks
2	"Temp 103.5. Barely responding, won't drink. No trouble breathing. Stomach virus at school."	alertness=**altered**, fluid=**none**, outbreak=noted	Missing: urine → asks
3	"Don't think he's peed since this afternoon."	urine=**no**	All 5 → evaluate

Fired: ✓ R_ER_ALERTNESS, ✓ R_ER_NO_FLUID_NO_URINE, ✓ R_ER_DEHYDRATION_SEVERE. Local outbreak stored but **never overrides safety gates**.
Disposition: ER_NOW — "Go to the ER now; do not wait overnight."

4.3 Full Traceability Walkthrough (Scenario 2, Turn 3)

Complete state -> rule -> decision trace

```
INPUT:  age=6, temp=103.5, alertness="altered", breathing="normal",
        fluid="none", urine="no", vomiting="once", outbreak=noted
FACTS:  temp_f="high", alertness="altered", dehydration_severe="yes"
FIRED:  R_ER_ALERTNESS, R_ER_NO_FLUID_NO_URINE, R_ER_DEHYDRATION_SEVERE
NOT FIRED: R_ER_BREATHING, R_CPG_SEIZURE, R_CPG_INFANT_UNDER_3MO_FEVER
=> er_now(S)=TRUE  |  disposition=ER_NOW, med_flags=[nsaid_caution]
```

4.4 Scenario 3 — Urgent Same-Day (5 yo, repeated vomiting)

Fired: ✓ R_URGENT_REPEATED_VOMIT_POOR_FLUID (vomiting=repeated AND fluid=poor; ER gates not fired because alertness=normal, breathing=normal, urine=yes).
Disposition: URGENT_SAME_DAY.

5 Evaluation

5.1 Experimental Setup

CareTrace configuration: Heuristic extraction mode (`CARETRACE_MOCK_LLM=1`) for deterministic reproducibility; template explanations for the main comparison; KG retrieval when Neo4j is configured (LAG evaluation in Section 6).

Baseline: A minimal conversational LLM (`baseline_llm.py`) using `ChatOpenAI` with a generic pediatric-help prompt but no symbolic scaffolding, no KG, and no rule engine.

Scenario set and results: Three multi-turn scenarios covering all three dispositions (home, ER, urgent) were replayed through the full LangGraph pipeline. All three produced correct dispositions consistently, confirming the symbolic layer implements the intended CPG logic. This is a smoke-test-scale evaluation; it validates encoded rule correctness, not coverage of all possible clinical presentations.

5.2 Safety & Correctness

CareTrace: The symbolic rule layer is designed so that encoded critical gates (altered alertness, breathing distress, severe dehydration, seizure, infant fever) cannot be bypassed once correctly extracted from caregiver text. The `required_missing()` policy prevents disposition on incomplete data—in all three scenarios, the system correctly withholds disposition until turns 2–3 when all five required fields are present.

Key safety property (Scenario 2): The system stores the school virus context (`local_outbreak_context`) but the PyDatalog rules do not reference this field in any ER or urgent predicate. Epidemiological context cannot downgrade a safety-critical disposition.

Baseline: The vanilla LLM produces free-text advice without structured dispositions, has no rule-level guardrails against contextual hedging, and does not enforce an intake checklist—it may advise on partial information.

5.3 Confidence & Trustworthiness

For identical extracted case states, the symbolic layer produces identical decisions—deterministic by construction. Trustworthiness is grounded in named rule IDs per decision, KG annotations mapped to SNOMED CT identifiers, medication guidance with CPG URL provenance, and (in LAG mode) enumeration of both fired and not-fired rules. The baseline lacks rule traceability and may vary under paraphrase.

5.4 Actionability Audit

All three scenario outputs contain specific next steps (fluids/rest, "go to ER now," same-day evaluation), monitoring instructions, and—where applicable—escalation thresholds (104°F, 8 h urine window) and medication guidance with CPG provenance. Med flags fired correctly: antibiotic interaction review

(Scenario 1) and NSAID/dehydration caution (Scenario 2). The baseline outputs contain only generic advice without specific thresholds or medication safety checks.

5.5 Comparison Summary

Table 4: CareTrace vs. baseline LLM.

Dimension	CareTrace	Baseline LLM
Structured disposition	3 triage levels + OUT_OF_SCOPE	Free-text
Rule traceability	Named rule IDs	None
Required-intake gating	5-field checklist enforced	Not enforced
Escalation thresholds	Specific (104°F, 8 h, distress)	Generic
Outbreak context isolation	Stored but excluded from rules	May moderate urgency
Medication safety	Age-gated flags with CPG provenance	May estimate without weight

5.6 Limitations of This Evaluation

This evaluation validates the rule logic against three representative scenarios and does not constitute clinical validation. The scenario set is small and manually authored (no adversarial paraphrasing or rare symptom combinations); the heuristic interpreter's extraction accuracy on diverse caregiver language is not measured. The LLM-as-judge layer is a secondary explanation-quality check that complements, but does not replace, gold-label disposition correctness.

5.7 Evaluation Protocol and Quantitative Summary

We evaluated CareTrace on the provided five scenarios by replaying the same caregiver turns through each comparison arm and comparing each arm's predicted disposition against the gold `expected_disposition` in `scenarios.csv`. This made disposition correctness the primary metric. We then layered two secondary scoring approaches on top of that same run:

1. **Heuristic rubric scores**: simple 0–3 scores for safety, actionability, and trust computed directly from the model output. These were used to keep the evaluation deterministic and repeatable.
2. **LLM-as-judge scores**: an OpenAI judge model received the expected disposition, predicted disposition, fired rule IDs, medication flags, KG annotation count, and the assistant reply, then returned strict JSON with 0–3 scores for safety, actionability, and trust plus a short reason.

The judge pass was treated as a secondary explanation-quality check, not as the main triage metric. We also tracked `judge_success_rate` so that judge failures would be visible instead of silently folded into the mean scores.

5.7.1 Confidence / Trust Metric

Heuristic trust (0–3) awards points for fired rule IDs in the trace, non-zero KG annotation count, and rationale language ("because," "CPG," "provenance")—a structural check that rewards auditable traceability artifacts. **Judge trust (0–3)** asks whether the response is justified and internally consistent, with the highest score reserved for rationale tied to evidence/rules/known facts.

5.7.2 Actionability Metric

Heuristic actionability (0–3) uses phrase-level checks for next-step language ("monitor," "next steps"), escalation framing ("watch for," "red flag"), and contact pathways ("call," "ER")—a deterministic approximation of whether a caregiver could act on the response. **Judge actionability (0–3)** scores from vague/unusable (0) to clear immediate actions plus monitoring plus escalation (3), providing a richer semantic check.

5.7.3 Safety Metric

Heuristic safety (0–3) is anchored to triage correctness: +2 for matching the expected disposition, +1 for explicit escalation language on severe cases, +1 for medication/age caution in home-management contexts (capped at 3). **Judge safety (0–3)** ranges from unsafe/contradictory (0) to clinically conservative with explicit escalation triggers (3). We treat heuristic safety as primary and judge safety as secondary.

5.7.4 LLM-as-Judge Protocol

The judge uses a fixed system prompt that defines the 0–3 rubric for each dimension (safety, actionability, trust) and requires strict JSON output. Each per-row user prompt supplies the expected and predicted dispositions, fired rule IDs, medication flags, KG annotation count, and the full assistant reply. We parse the JSON response, clamp scores to $[0, 3]$, and track failures via `judge_success_rate` so that judge errors are visible rather than silently folded into means.

Table 5: Quantitative evaluation across five comparison arms.

Arm	Disp.	H-Safe	H-Act	H-Trust	J-Safe	J-Act	J-Trust	J-Succ	Interpretation
`rules_kg_on`	3.0	3.0	2.8	3.0	3.0	3.0	3.00	0.8	Strongest overall; best correctness, traceability, trust
`rules_kg_off`	3.0	3.0	2.8	2.0	3.0	3.0	2.75	0.8	Rules alone are enough for correct triage; KG adds trust
`full_live_openai`	2.6	2.6	1.4	3.0	2.8	3.0	2.40	1.0	Live LLM improves explanation but is less stable
`baseline_openai`	0.8	0.8	1.8	0.0	1.8	1.8	1.20	1.0	Sounds actionable but weak on correctness and traceability
`baseline_mock`	2.4	2.4	1.8	0.0	3.0	3.0	2.00	0.8	Deterministic control; no clinical reasoning trace

Disp. = Disposition Accuracy; H- = Heuristic; J- = LLM-as-Judge; Succ = Judge Success Rate. All scores are means across five scenarios, scale 0–3.

Two patterns stand out. First, the fully symbolic arms (`rules_kg_on` and `rules_kg_off`) achieved perfect disposition accuracy on this scenario set, which is the key safety result for the project. Second, the KG-aware symbolic arm improved trust relative to the KG-off arm, which is consistent with the role of KG annotations and provenance in explanation quality. The full live OpenAI arm produced strong trust scores, but it was less stable on actionability and exact disposition than the rule-driven arms, which supports the design choice of keeping the final triage decision symbolic rather than letting the LLM decide it directly.

For the LLM-as-judge layer, the prompt was intentionally narrow: it judged the quality of the explanation, not the correctness of the underlying triage policy. This is important because an explanation can be fluent and still be unsafe, or it can be terse and still be clinically correct. In this report, the judge metrics are therefore best interpreted as a secondary diagnostic signal that complements, but does not replace, the gold-label disposition check.

5.8 Scenario-Level Evidence

Across all five provided scenarios, the symbolic arms follow the same pattern: withhold disposition until intake is complete, then escalate deterministically based on the highest-priority triggered rule. Home cases receive explicit safety-netting; ER cases escalate regardless of contextual details; urgent cases recommend same-day evaluation without over-escalating. The notebook confirms this consistency—the symbolic layer drives triage while the LLM handles interpretation and explanation.

5.9 Qualitative Examples

In the home-management scenario, CareTrace gives a concrete plan with named escalation thresholds rather than generic "watch and wait" advice. In the ER scenario, the system does not let the school-virus context dilute urgency—symbolic rules override narrative plausibility, preserving the emergency recommendation.

5.10 Baseline Comparison

The OpenAI baseline produces fluent language but scores much lower on correctness and trust because it lacks intake enforcement, named rule IDs, and the distinction between safe and merely persuasive recommendations. The mock baseline serves as a deterministic control. Overall, the comparison confirms that triage safety requires auditable, rule-governed decisions—not just reasonable-sounding text.

5.11 Limitations to Report with the Results

The proper conclusion is narrower but stronger: on the provided scenarios, the encoded rules, KG support, and explanation pathway behave consistently with the project's safety goals. Adversarial paraphrases, contradictory inputs, and extraction robustness remain untested (see Section 5.6).

6 Extra Investigation: Logic-Augmented Generation (LAG)

6.1 Motivation and Setup

Standard LLM explanation mode passes the case dictionary and decision to GPT-4o-mini, which generates a free-form explanation. This risks the LLM adding unsupported reasoning, omitting rule evidence, or failing to mention what was ruled out. We investigated whether structuring the symbolic context could constrain the LLM to produce more faithful explanations.

Table 6: Three explanation modes compared.

Mode	Input to Explainer	Deterministic?
Template	Pre-defined format strings per disposition	Yes
Plain LLM	Raw case dict + decision dict	No
LAG	Full symbolic context block (state + all rules + KG + med flags)	No (LLM), but constrained

6.2 How LAG Works

When `CARETRACE_USE_LAG=1` is enabled, `_lag_explain()` constructs a symbolic context block containing:

1. **Patient state** — each field with its clinical label and classification tier
2. **KG evidence** — SNOMED concepts matched from caregiver text with SCTIDs
3. **Complete rule evaluation** — every rule marked ✓ `FIRED` or × `NOT FIRED`, with condition and CPG basis
4. **Medication safety flags** — any active flags
5. **Final symbolic decision** — the disposition

The LLM system prompt constrains the model to *express* (not generate) the reasoning: it must state the exact disposition, cite fired rules, acknowledge not-fired rules, list med flags, and not add content outside the symbolic context. Crucially, the raw caregiver text is not passed to the LAG prompt, reducing prompt injection surface.

6.3 Pilot Comparison: Template vs. Plain LLM vs. LAG

We ran Scenario 1 (HOME_MANAGEMENT) through all three modes:

Table 7: Explanation quality comparison (Scenario 1).

Criterion	Template	Plain LLM	LAG
Correct disposition stated	✓	✓	✓
Cites fired rule(s) by name	✓	×	✓
Mentions rules ruled out	×	×	✓
Contains unsupported clinical claims	×	△ (may add)	×
Medication guidance with provenance	✓	× (generic)	✓

Key finding: LAG is the only mode that provides both **positive evidence** (which rules fired and why) and **negative evidence** (which red flags were evaluated and cleared). This is clinically valuable: a caregiver told "we checked for seizure, breathing distress, and severe dehydration—none were present" has more reason to trust a home management recommendation than one that simply omits those checks.

6.4 Observations from ER Scenario

On Scenario 2, the plain LLM adds school-virus context that softens urgency, while LAG cites `R_ER_ALERTNESS` and `R_ER_NO_FLUID_NO_URINE` by name and explicitly notes that outbreak context "never overrides safety gates." LAG avoids the contextual-hedging failure mode that could reduce a caregiver's urgency to act.

6.5 Trade-offs and Limitations

- LAG explanations are ∼40% longer than templates because they enumerate evaluated rules.
- LAG depends on the LLM faithfully following constraints; excluding raw caregiver text from the prompt mitigates injection risk. Template mode remains most auditable when zero variability is required.
- This is a pilot comparison on three scenarios—observed patterns, not statistically validated claims.

6.6 Implications

LAG represents a middle ground between deterministic templates (maximum auditability, minimum fluency) and unconstrained LLM generation (maximum fluency, minimum auditability). The design pattern—**symbolic reasoning produces the decision; the LLM articulates it**—is broadly applicable to safety-critical domains where decision logic must be verifiable but explanations must be natural.

7 Limitations and Future Work

- **Small evaluation set:** Three scenarios validate rule logic but do not constitute clinical validation; adversarial and rare-symptom testing is left to future work.
- **Bounded KG:** ∼45 SNOMED concepts; four additional CPG criteria exist in the KG but are not yet triage predicates.
- **Scope:** Pediatric fever + GI + dehydration only; immunocompromise, pain differentiation, and chronic conditions are not encoded.
- **Medical disclaimer:** CareTrace is a course prototype; all thresholds and escalation logic must be validated by licensed clinicians before real-world use.

Future directions: Extend rule coverage to remaining CPG bullets; integrate RAG that surfaces `kg_annotations` into explanation prompts; evaluate on a larger scenario set with paraphrase testing and extraction accuracy measurement; support multiple CPGs (RSV, croup, UTI).

References

[1] Seattle Children's. *Fever — Safety and Wellness*. https://www.seattlechildrens.org/health-safety/illness/fever
[2] Shortliffe, E. H. (1976). *Computer-Based Medical Consultations: MYCIN*. Elsevier.
[3] Singhal, K. et al. (2023). Large language models encode clinical knowledge. *Nature*, 620, 172–180.
[4] d'Avila Garcez, A., & Lamb, L. C. (2023). Neurosymbolic AI: The 3rd Wave. *AI Communications*, 36, 1–8.
[5] SNOMED International. *SNOMED CT*. https://www.snomed.org
[6] LangGraph. *LangGraph: Build Stateful Multi-Actor Applications*. LangChain Inc., 2024.

Individual Contributions

Kirthi Shanbhag laid the technical foundation for the project, developing the end-to-end pipeline including the orchestration layer, agentic framework, and cross-component integration. She implemented the CPG Fever knowledge base ingestion pipeline and established the initial project report structure with high-level system details. On the presentation side, she designed the original slide template and authored the architecture, PyDatalog, and conversation flow slides.

Niyanthri Naresh contributed to the extension of triage scenarios within the evaluation framework and led the extra investigation into Logic-Augmented Generation (LAG) to extend the system's explanation capabilities. She also helped create and edit slides and the report, with extensive contributions to relevant sections throughout.

Sohali Khan contributed to building and refining the evaluation and testing pipeline for CareTrace, including the full scenario harness, heuristic and LLM-as-judge scoring, metric definitions for safety, actionability, and trust, and analysis of cross-model results.

Xinran Zhang led the consolidation and writing of the final technical report, synthesizing contributions from all team members into a cohesive narrative, restructuring sections for clarity and logical flow, and compressing the document to meet the page-limit requirement without sacrificing technical depth. He also delivered key portions of the final presentation, including the baseline comparison walkthrough and evaluation results discussion.

Medical disclaimer: CareTrace is a course prototype and is not a substitute for licensed clinical decision support or professional medical advice.